

Compilation

JavaCC

Nicolas Delestre

Présentation générale

- JavaCC (*Java Compiler Compiler* <https://javacc.org/>) permet de développer des interpréteurs et des compilateurs avec le langage Java
- JavaCC est un compilateur :
 - il prend en entrée un fichier (d'extension .jj) décrivant les unités lexicales, la grammaire attribuée et les schémas de traduction en Java associé à chaque règle
 - il génère l'analyseur lexical et l'analyseur syntaxique

Structure d'un fichier .jj

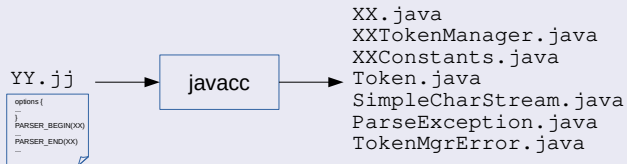
Pour la création de l'analyseur syntaxique qui se nommera XX

```
options {  
    // configuration de l'analyseur lexical et syntaxique  
}  
  
PARSER_BEGIN(XX)  
    //description de la classe XX : l'analyseur syntaxique  
PARSER_END(XX)  
  
SKIP : {  
    //liste des caractères qui seront écartés par l'analyseur lexical  
}  
  
TOKEN : {  
    // liste des unités lexicales <uniteLexicale,expressionRationnelle>  
}
```

Une "méthode" par non terminal qui sera ajoutée à la classe XX
Composée de deux parties:

- 1 - Signature + déclaration
- 2 - Les règles de production avec le code Java associé tel que:
 - les unités lexicales sont entourées de < et >
 - l'utilisation des non terminaux correspondent aux appels de méthodes

Classes générées



- `XX.java` : l'analyseur syntaxique
- `XXTokenManager.java` : l'analyseur lexical
- `XXConstants.java` : les unités lexicales partagées par l'analyseur lexical et l'analyseur syntaxique
- des classes indépendantes de la grammaire : `Token.java`, `SimpleCharStream.java`, `ParseException.java`, `SimpleCharStream.java`

Un exemple d'interpréteur : une calculatrice à mémoire

Fonctionnement

```
$java -cp classes fr.insarouen.iti.compilation.calculatrice.Main  
> 2*(3+4)/(4-8)  
-3.5  
> let a=2*(3+4)/(4-8)  
> a  
-3.5  
> 2*a  
-7.0  
> let A=2*8-1  
> A  
15  
> hex(a)  
f  
> bin(a)  
1111  
> exit  
$
```

- Les calculs sont indépendants les uns des autres, l'analyseur syntaxique analyse à chaque fois une seule action qui peut être :
 - l'affichage (décimal par défaut, binaire, hexadécimal) de la valeur d'une expression (cela nécessite une mémoire)
 - l'affectation de la valeur d'une expression à une variable (modification de la mémoire)
 - la sortie du programme
- Les variables seront stockées dans une mémoire (qui associe un identifiant à une valeur)

Les unités lexicales (et leurs expressions rationnelles)

- Unités lexicales liées aux expressions arithmétiques : PLUS (+), MOINS (-), MUL (*), DIV(/), PARENG(() , PAREND())
- Unité lexicale BIN (bin)
- Unité lexicale HEX (hex)
- Unité lexicale AFF (=)
- Unité lexicale LET (let)
- Unité lexicale SORTIR (exit)
- Unité lexicale ID ($[A-Z, a-z, _][A-Z, a-z, _, 0-9]^*$)
- Unité lexicale NB_ENTIER ($[0-9]^+$)
- Unité lexicale NB_REEL ($[0-9]^+.[0-9]^*$)

La grammaire initiale

<i>Action</i>	→	<i>Expression</i> <i>BIN PARENG Expression PAREND</i> <i>HEX PARENG Expression PAREND</i> <i>Affectation</i> <i>SORTIR</i>
<i>Affectation</i>	→	<i>LET ID AFF Expression</i>
<i>Expression</i>	→	<i>Expression PLUS Expression</i> <i>Expression MOINS Expression</i> <i>Expression MUL Expression</i> <i>Expression DIV Expression</i> <i>PLUS Expression</i> <i>MOINS Expression</i> <i>PARENG Expression PAREND</i> <i>NB_ENTIER</i> <i>NB_REEL</i> <i>ID</i>

Grammaire ambiguë

- JavaCC ne permet pas de fixer des priorités ou des associativités aux unités lexicales
- On doit modifier la grammaire en décomposant une expression en terme et facteur

La grammaire en séparant expression, terme et facteur

Action	→	<i>Expression</i> <i>BIN PARENG Expression PAREND</i> <i>HEX PARENG Expression PAREND</i> <i>Affectation</i> <i>SORTIR</i>
Affectation	→	<i>LET ID AFF Expression</i>
Expression	→	<i>Terme</i> <i>Expression PLUS Terme</i> <i>Expression MOINS Terme</i>
Terme	→	<i>Facteur</i> <i>Terme MUL Facteur</i> <i>Terme DIV Facteur</i>
Facteur	→	<i>Element</i> <i>PLUS Element</i> <i>MOINS Element</i>
Element	→	<i>NB_ENTIER</i> <i>NB_REEL</i> <i>ID</i> <i>PARENG Expression PAREND</i>

JavaCC est $LL(k)$

- JavaCC n'accepte pas la récursivité à gauche
- On doit modifier la grammaire en transformant toute règle $A \rightarrow A\alpha \mid \beta$ en $A \rightarrow \beta A'$ et $A' \rightarrow \alpha A' \mid \epsilon$

La grammaire en supprimant les récursivités à gauche

<i>Action</i>	$\rightarrow$	<i>Expression</i> <i>BIN PARENG Expression PAREND</i> <i>HEX PARENG Expression PAREND</i> <i>Affectation</i> <i>SORTIR</i>
<i>Affectation</i>	$\rightarrow$	<i>LET ID AFF Expression</i>
<i>Expression</i>	$\rightarrow$	<i>Terme Expression'</i>
<i>Expression'</i>	$\rightarrow$	<i>PLUS Terme Expression'</i> <i>MOINS Terme Expression'</i> ϵ
<i>Terme</i>	$\rightarrow$	<i>Facteur Terme'</i>
<i>Terme'</i>	$\rightarrow$	<i>MUL Facteur Terme'</i> <i>DIV Facteur Terme'</i> ϵ
<i>Facteur</i>	$\rightarrow$	<i>Element</i> <i>PLUS Element</i> <i>MOINS Element</i>
<i>Element</i>	$\rightarrow$	<i>NB_ENTIER</i> <i>NB_REEL</i> <i>ID</i> <i>PARENG Expression PAREND</i>

Cas du ϵ

- JavaCC ne permet d'exprimer le ϵ dans la partie droite d'une règle
- Mais JavaCC permet d'utiliser l'opérateur de fermeture de Kleene
- On doit modifier la grammaire en remplaçant des couples de règles $A \rightarrow \alpha B$ et $B \rightarrow \beta B \mid \epsilon$ en $A \rightarrow \alpha\beta^*$

La grammaire en utilisant la fermeture de Kleene

Action	→	Expression BIN PARENG Expression PAREND HEX PARENG Expression PAREND Affectation SORTIR
Affectation	→	LET ID AFF Expression
Expression	→	Terme ((PLUS MOINS) Terme)*
Terme	→	Facteur ((MUL DIV) Facteur)*
Facteur	→	Element PLUS Element MOINS Element
Element	→	NB_ENTIER NB_REEL ID PARENG Expression PAREND

- Il n'y a plus d'ambiguïté, ni de conflit : LOOKAHEAD=1
- On veut un seul objet pour toutes les lignes d'interprétation : STATIC=true
- Non sensible à la casse : IGNORE_CASE=true

Paramétrage de JavaCC

```
1 options {  
2   LOOKAHEAD = 1;  
3   STATIC = true;  
4   IGNORE_CASE = true;  
5 }
```

La classe

```
7 _PARSER_BEGIN(AnalyseurSyntaxique)
8  package fr.insarouen.iti.compilation.calculatrice.analyseurSyntaxique;
9
10 import fr.insarouen.iti.compilation.calculatrice.tableDesSymboles.Identifiant;
11 import fr.insarouen.iti.compilation.calculatrice.ast.RepresentationDeNaturel;
12 import fr.insarouen.iti.compilation.calculatrice.ast.Addition;
13 import fr.insarouen.iti.compilation.calculatrice.ast.Soustraction;
14 import fr.insarouen.iti.compilation.calculatrice.ast.Multiplication;
15 import fr.insarouen.iti.compilation.calculatrice.ast.Division;
16 import fr.insarouen.iti.compilation.calculatrice.ast.Quitter;
17 import fr.insarouen.iti.compilation.calculatrice.ast.Affecter;
18 import fr.insarouen.iti.compilation.calculatrice.ast.Calculer;
19 import fr.insarouen.iti.compilation.calculatrice.ast.Expression;
20 import fr.insarouen.iti.compilation.calculatrice.ast.Action;
21 import fr.insarouen.iti.compilation.calculatrice.ast.Constante;
22 import fr.insarouen.iti.compilation.calculatrice.ast.Variable;
23 import fr.insarouen.iti.compilation.calculatrice.ast.Moins;
24 import fr.insarouen.iti.compilation.calculatrice.exceptions.IdentifiantException;
25 public class AnalyseurSyntaxique {
26
27 }
28 _PARSER_END(AnalyseurSyntaxique)
```

Les unités lexicales

```
30 SKIP : {
31     " "
32     | "\t"
33 }
34
35 TOKEN:{
36     <PLUS: "+">
37     | <MOINS: "-">
38     | <MUL: "*">
39     | <DIV: "/">
40     | <ALALIGNE: "\n">
41     | <PARENG: "(">
42     | <PAREND: ")">
43     | <AFF: "=">
44     | <LET: "let">
45     | <SORTIR: "exit">
46     | <HEX: "hex">
47     | <BIN: "bin">
48     | <#CHIFFRE: ["0"-"9"]>
49     | <NB_ENTIER: (<CHIFFRE>)+>
50     | <NB_REEL: <NB_ENTIER> "." (<CHIFFRE>)*>
51     | <ID: ["a"-"z", "_"] (["a"-"z", "_", "0"-"9"])*>
52 }
```

L'axiome

```

54 Action action() : {
55     Expression exp; Token id;
56 }
57 {
58     (
59         exp = expression() { /* Action -> Expression */
60             return new Calculer(exp, RepresentationDeNaturel.DECIMALE);
61         }
62         | <BIN> <PARENG> exp = expression() <PAREND> { /* Action -> BIN PARENG Expression PAREND */
63             return new Calculer(exp, RepresentationDeNaturel.BINAIRE);
64         }
65         | <HEX> <PARENG> exp = expression() <PAREND> { /* Action -> HEX PARENG Expression PAREND */
66             return new Calculer(exp, RepresentationDeNaturel.HEXADECIMALE);
67         }
68         | <LET> id = <ID> <AFF> exp = expression() { /* Action -> Affectation, Affectation -> LET ID AFF Expression */
69             Affecter aff = null;
70             try {
71                 aff = new Affecter(new Identifiant(id.image), exp);
72             } catch (IdentifiantException e) {
73                 // impossible car l'analyseur lexical vérifie ces conditions
74             }
75             return aff;
76         }
77         | <SORTIR> { /* Action -> SORTIR */
78             return new Quitter();
79         }
80     ) <ALALIGNE>
81 }

```

Les expressions et termes

```
83 Expression expression() : {
84     Expression op, res;
85 }
86 {
87     res = terme()
88     (
89         <PLUS> op = terme() {res = new Addition(res, op);}
90         | <MOINS> op = terme() {res = new Soustraction(res, op);}
91     )* {return res;}
92 }
93 }
94
95
96 Expression terme() : {
97     Expression op, res;
98 }
99 {
100     res = facteur()
101     (
102         <MUL> op = facteur() {res = new Multiplication(res, op);}
103         | <DIV> op = facteur() {res = new Division(res, op);}
104     )* {return res;}
105 }
```

Les facteurs

```
107 Expression facteur() : {
108     Expression res;
109 }
110 {
111     res = element() {return res;}
112     | <PLUS> res = facteur() {return res;}
113     | <MOINS> res = facteur() {return new Moins(res);}
114 }
```


Les facteurs et éléments

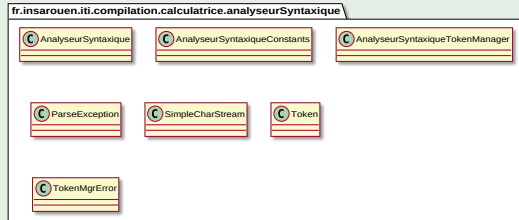
```
116 Expression element() : {
117     Token t;
118     Expression exp, res;
119 }
120 {
121     t = <NB_ENTIER> {
122         try {
123             return new Constante(Integer.parseInt(t.image));
124         } catch (NumberFormatException e) { } // impossible car l'analyseur lexical vérifie ces conditions
125     }
126     | t = <NB_REEL> {
127         try {
128             return new Constante(Float.parseFloat(t.image));
129         } catch (NumberFormatException e) { } // impossible car l'analyseur lexical vérifie ces conditions
130     }
131     | t = <ID> {
132         try {
133             return new Variable(new Identifiant(t.image));
134         } catch (IdentifiantException e) { } // impossible car l'analyseur lexical vérifie ces conditions
135     }
136     | <PARENG> res = expression() <PAREND> { return res; }
137 }
```

Compilation du fichier .jj

Compilation des analyseurs syntaxiques et sémantiques

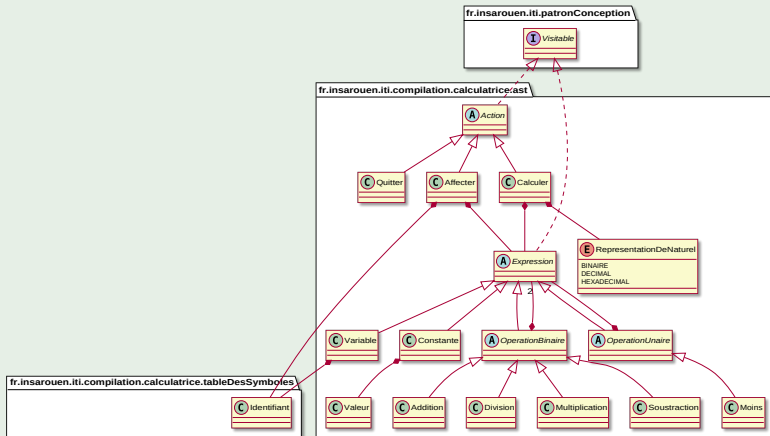
```
analyseurSyntaxique$ javacc AnalyseurSyntaxique.jj
Java Compiler Compiler Version 5.0 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file AnalyseurSyntaxique.jj . . .
File "TokenMgrError.java" is being rebuilt.
File "ParseException.java" is being rebuilt.
File "Token.java" is being rebuilt.
File "SimpleCharStream.java" is being rebuilt.
Parser generated successfully.
```

Diagramme UML des classes produites



Les classes de l'AST

Diagramme UML des classes de l'AST



Exemple d'AST

Diagramme UML d'objets de l'AST créé à partir de l'instruction : `let a=2*b`

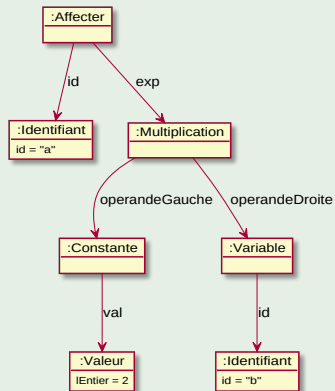
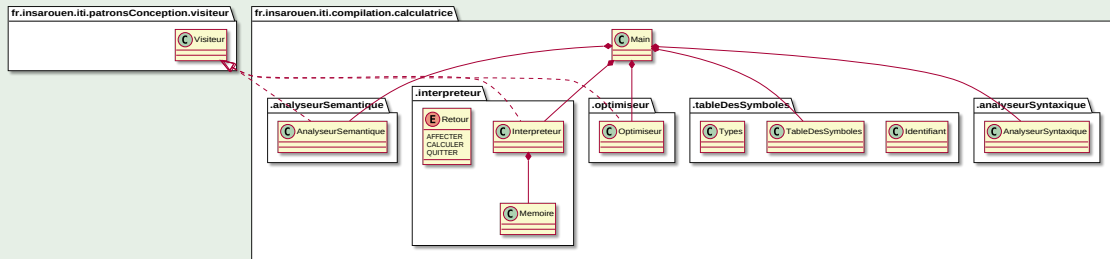


Diagramme UML



La classe Main 2 / 2

Programme principal

```
1 package fr.insarouen.iti.compilation.calculatrice;
...
19 public class Main {
20     private static final Logger LOGGER = Logger.getLogger("Calculatrice");
21
22     public static void main(String args[]) {
23         TableDesSymboles tds = new TableDesSymboles();
24         Memoire mem = new Memoire();
25         Interpreteur interpreteur = new Interpreteur(mem, tds);
26         AnalyseurSyntaxique analyseurSyntaxique = new AnalyseurSyntaxique(System.
27             in);
28         Action action;
29         Retour res;
30         boolean quitterLeProgramme = false;
31
32         LOGGER.setLevel(Level.OFF);
33         if (args.length == 1)
34             if (args[0].equals("-v")) {
35                 System.setProperty("java.util.logging.SimpleFormatter.format",
36                     "%5$s %n");
37                 LOGGER.setLevel(Level.INFO);
38             }
39         do {
40
41             ...
42
43             } while (!quitterLeProgramme);
44     }
45 }
46
47 try {
48     System.out.print("> ");
49     action = analyseurSyntaxique.action();
50     LOGGER.info(" Après saisie : "+action.toString());
51     action.accepter(new AnalyseurSemantique(tds));
52     action = new Optimiseur().optimiser(action);
53     LOGGER.info(" Après optimisation : "+action.toString());
54     res = interpreteur.interpreter(action);
55     switch (res) {
56         case CALCULER :
57             System.out.println(interpreteur.getResultatDuCalcul());
58             break;
59         case QUITTER :
60             quitterLeProgramme = true;
61             break;
62     }
63 } catch (RepresentationException e) {
64     System.err.println("Erreur sémantique : "+e.getMessage());
65 } catch (IdentifiantInconnuException e) {
66     System.err.println("Erreur d'interprétation : "+e);
67 } catch (CalculatriceException e) {
68     System.err.println(e);
69 } catch (ArithmeticException e) {
70     System.err.println("Erreur arithmétique : "+e);
71 } catch (ParseException | TokenMgrError e) {
72     System.err.println("Erreur de syntaxe");
73 } catch (Throwable e) {
74     System.err.println(e);
75 } finally {
76     analyseurSyntaxique.ReInit(System.in);
77 }
```

Implantation du patron de conception Visiteur

Exemple de traitement de l'AST : AnalyseurSemantique

```
26 public class AnalyseurSemantique implements Visiteur {
27     TableDesSymboles tds;
28     Stack<Types> typesRencontres;
29
30     public AnalyseurSemantique(TableDesSymboles tds) {
31         this.tds = tds;
32         typesRencontres = new Stack();
33     }
34
35     public void visiter(Constante constante) throws Throwable {
36         typesRencontres.push(constante.getVal().getType());
37     }
38
39     public void visiter(Variable variable) throws Throwable {
40         typesRencontres.push(tds.getType(variable.getId()));
41     }
42
43     public void visiter(OperationBinaire exp) throws Throwable {
44         exp.getOperandeGauche().accepter(this);
45         exp.getOperandeDroite().accepter(this);
46         Types tOpG, tOpD;
47         tOpD = typesRencontres.pop();
48         tOpG = typesRencontres.pop();
49         if (tOpG == Types.REEL || tOpD == Types.REEL)
50             typesRencontres.push(Types.REEL);
51         else
52             typesRencontres.push(Types.ENTIER);
53     }
```

...

```
64     public void visiter(Calculer calculer) throws Throwable {
65         if (calculer.getReprentation() == RepresentationDeNaturel.BINAIRE
66             ||
67             calculer.getReprentation() == RepresentationDeNaturel.
68                 HEXADECIMALE) {
69             calculer.getExpression().accepter(this);
70             if (typesRencontres.pop() != Types.ENTIER) {
71                 String raison = "seuls les naturels peuvent être représentés
72                     en " +
73                     ((calculer.getReprentation() == RepresentationDeNaturel.
74                         BINAIRE) ? "binaire": "hexadécimale");
75                 throw new RepresentationException(raison);
76             }
77         }
78     }
79
80     public void visiter(Action action) throws Throwable {
81         // Rien à faire si l'action est QUITTER ou AFFECTER
82     }
```

Conclusion

- Dans ce cours nous avons vu que :
 - JavaCC permet de créer assez facilement des compilateurs ou des interpréteurs en Java
 - Le framework JavaCC propose aussi :
 - JJTree un préprocesseur à JavaCC (l'outil), un tutoriel vidéo à JJTree :
<https://www.youtube.com/watch?v=9V4GzEomc5w>
 - JJDoc un générateur de documentation de grammaire au format BNF (*Backus Naur Form*)